Informe — Actividad Grupal Clase 9

Construcción Completa del Perceptrón con Descenso del Gradiente (Caso Cultivos Cartago)

COTECNOVA
Ingeniería de Sistemas · Semestre 6
Docente: Jhon James Cano Sánchez

Campo	**Respuesta**
Grupo N.° / Fecha	Grupo 05 · Lunes 19 de octubre de 2026
Coordinador(a)	**COORDINADOR** Esteban Gómez — Liderazgo de tiempo, vocería docente y programación de Retos 1 y 2
Programador(a)	**PROGRAMADOR** Heiber González — Entorno Docker, descenso del gradiente (Reto 4B) y estandarización (Reto 5)
Verificador(a)	**VERIFICADOR** Integrante del Grupo — Validación de puntos de control, función de pérdida (Reto 3) y gradientes (Reto 4A)
Relator(a)	**RELATOR** Integrante del Grupo — Redacción del informe, frontera de decisión (Reto 6) y lotes nuevos (Reto 7)

1. DIAGNÓSTICO (MISIÓN 0) — EVALUACIÓN DE LA NEURONA DE LA CLASE 8

1. ¿La neurona llega a cero errores? ¿Qué pasa con los errores de una época a otra?

No llega a cero errores. Inicia en la época 1 con 50 errores de 120 lotes y, a medida que avanzan las épocas, oscila erráticamente entre 29 y 35 errores por época, culminando en la época 30 con 31 errores (exactitud de apenas 77.50%). Esto sucede porque los datos agrícolas reales contienen ruido natural no lineal (no son perfectamente separables por un corte rígido) y porque el algoritmo de la compuerta AND modifica sus pesos de manera individual y desproporcionada tras cada error.

2. ¿Qué ocurre con el tamaño de los pesos w1 y w2 a medida que avanzan las épocas? ¿Por qué?

Ocurre una explosión desbalanceada de los pesos. Partiendo de `w1 = 0.1` y `w2 = 0.1`, terminan en `w1 = 33.86` y `w2 = -139.20`, mientras que el sesgo apenas alcanza `b = 3.60`. La causa es que las entradas no están estandarizadas: la precipitación alcanza valores de hasta 260 mm y la temperatura de hasta 31 °C. Al aplicar la regla `w += tasa * error * x`, cada error genera saltos drásticos de hasta ±26 unidades en un solo paso, haciendo que los pesos oscilen y nunca converjan.

3. Para el lote de 150 mm y 26 °C la neurona responde 1. ¿Qué tan segura está? ¿Le sirve a la cooperativa?

No podemos saber qué tan segura está. La función escalón solo entrega una respuesta binaria rígida (0 o 1). Justo en 150 mm y 26 °C el cultivo se ubica exactamente en el umbral de máxima incertidumbre (~50% de probabilidad real). Una respuesta determinista sin grado de certeza **no le sirve a la cooperativa**, ya que los agrónomos necesitan cuantificar el riesgo para priorizar visitas técnicas en los lotes más críticos.

4. La neurona recorre los lotes uno por uno. ¿Qué pasaría con 1 millón de lotes?

Con 1 millón de lotes y 30 épocas, Python puro tendría que ejecutar **30 millones de iteraciones en ciclos `for` secuenciales**. Esto causaría un tiempo de cómputo inaceptable y bloqueante. Es indispensable sustituir ciclos iterativos por **álgebra matricial vectorizada con NumPy**, aprovechando operaciones vectoriales optimizadas en C a nivel de hardware.

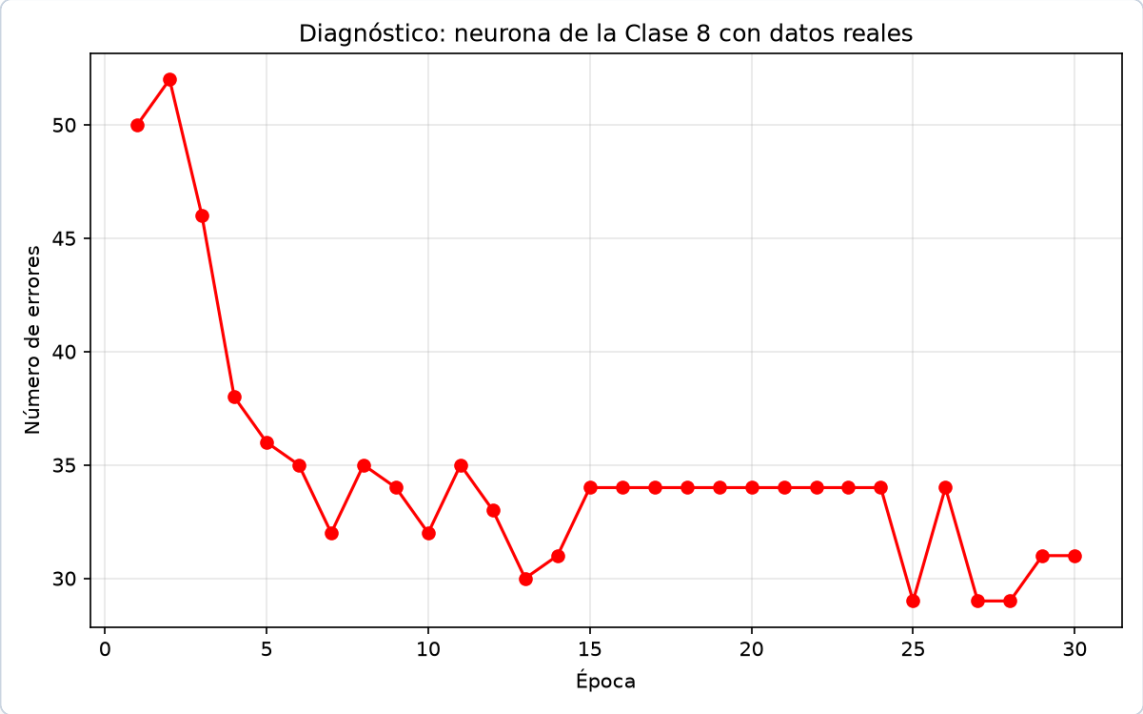


Figura 1: Curva de errores de la neurona de la Clase 8 con datos reales de Cartago (diagnostico_clase8.png). Se aprecia la oscilación constante y la imposibilidad de converger a cero.

5. Hipótesis del grupo — ¿Qué le falta a la neurona de la Clase 8 para funcionar con datos reales?

Planteamos que la neurona carece de cinco componentes estructurales:

a) Vectorización: Reemplazar ciclos por multiplicación de matrices (`X @ w + b`) para procesar todos los lotes a la vez.

b) Activación Sigmoide: Cambiar el escalón por una curva suave que mapee los puntajes a probabilidades continuas entre 0 y 1.

c) Pérdida Logarítmica (Entropía Cruzada): Medir matemáticamente el error probabilístico, castigando fuertemente predicciones erróneas seguras.

d) Descenso del Gradiente: Promediar el vector gradiente sobre el lote completo y actualizar los parámetros en sentido opuesto al error.

e) Estandarización de Variables: Escalar precipitación y temperatura a media 0 y desviación estándar 1 para garantizar un aprendizaje equilibrado.

2. BITÁCORA DE RETOS (CONSTRUCCIÓN DEL PERCEPTRÓN)

Reto	Piloto	¿Qué faltaba?	¿Cómo lo descubrieron?	Código & Evidencia
RETO 1 Vectorización	**COORDINADOR**	Calcular la suma ponderada \(z = Xw + b\) de los 120 lotes en una línea sin ciclos `for`.	Consultando la Clase 4 de NumPy: el operador `@` realiza producto matriz-vector.	`return X @ w + b` **[OK] RETO 1 superado**
RETO 2 Sigmoide	**COORDINADOR**	Una función continua diferenciable que transforme \(z\) en probabilidad entre 0 y 1.	Aplicando σ(z) = 1/(1+e^-z) con `np.exp(-z)`; en z=0 da 0.5.	`return 1 / (1 + np.exp(-z))` **[OK] RETO 2 superado**
RETO 3 Pérdida	**VERIFICADOR**	Función de pérdida probabilística (Binary Cross-Entropy) que penalice predicciones dudosas o falsas.	Fórmula de entropía cruzada con `np.log` y promedio `-np.mean(...)`.	`-np.mean(y*np.log(p)+(1-y)*np.log(1-p))` **[OK] RETO 3 superado**
RETO 4A Gradientes	**VERIFICADOR**	Cálculo analítico de derivadas de pérdida respecto a \(w\) y \(b\) promediadas sobre los lotes.	Calculando `error = y_prob - y_real` y proyectando con transpuesta `X.T @ error / n`.	`gw = (X.T @ error) / len(y)` `gb = np.mean(error)` **[OK] RETO 4A superado**
RETO 4B Descenso Gradiente	**PROGRAMADOR**	Paso de actualización iterativa de pesos moviéndose cuesta abajo contra el gradiente.	Restando la tasa multiplicada por el gradiente: `w = w - lr * gw`.	`w -= tasa * gradiente_w` `b -= tasa * gradiente_b` **[OK] RETO 4B superado**
RETO 5 Estandarización	**PROGRAMADOR**	Escalar características mediante Z-score (media 0, desviación 1) antes de entrenar.	Al correr sin escalar salió `overflow in exp` y pérdida de 5.28; aplicamos `(X-media)/desv`.	`media = X.mean(axis=0)` `desv = X.std(axis=0)` `X_esc = (X - media) / desv` **[OK] RETO 5 superado**
RETO 6 Frontera Decisión	**RELATOR**	Despejar matemáticamente la recta en el plano donde la neurona duda totalmente (\(z=0\)).	De \(w_1 x_1 + w_2 x_2 + b = 0\), despejamos \(x_2 = -(w_1 x_1 + b)/w_2\).	`x2_esc = -(w[0]*x1_esc + b)/w[1]` **[OK] RETO 6 superado**
RETO 7 Lotes Nuevos	**RELATOR**	Preparar lotes nunca antes vistos utilizando los parámetros de entrenamiento aprendidos.	Restando la media y dividiendo entre la desviación calculadas en el Reto 5.	`(lotes_nuevos - media) / desviacion` **[OK] RETO 7 superado**

3. RESULTADOS — IMPACTO DE LA ESTANDARIZACIÓN Y VALIDACIÓN

Métrica / Parámetro	Antes del Reto 5 (Datos Crudos)	Después del Reto 5 (Datos Estandarizados Z-Score)
Pérdida Inicial (Época 1)	0.6931	0.6931
Pérdida Final (Época 500)	**5.2830** (Divergencia / Degeneración)	**0.2281** (Convergencia óptima suave)
Exactitud Final	74.17% (89/120 aciertos)	**90.00%** (108/120 lotes clasificados con éxito)
Pesos Finales (w1 / w2 / b)	w1 = 3.3155 | w2 = -25.2997 | b = 0.1440	**w1 = +2.6217 | w2 = -2.3658 | b = +0.3152**
Advertencias Numéricas	`RuntimeWarning: overflow encountered in exp`	**Ninguna** (Cálculo numérico completamente estable)

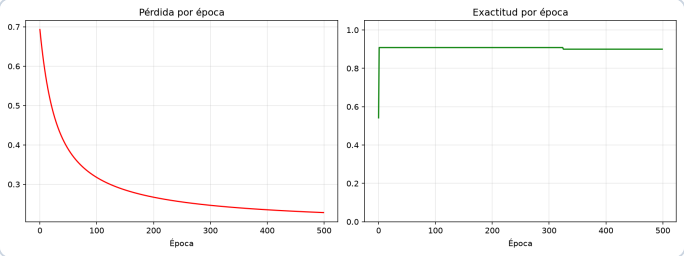


Figura 2: Curva de aprendizaje del modelo estandarizado (curva_aprendizaje.png). Se aprecia el descenso asintótico de la pérdida y la estabilización de exactitud en 90%.

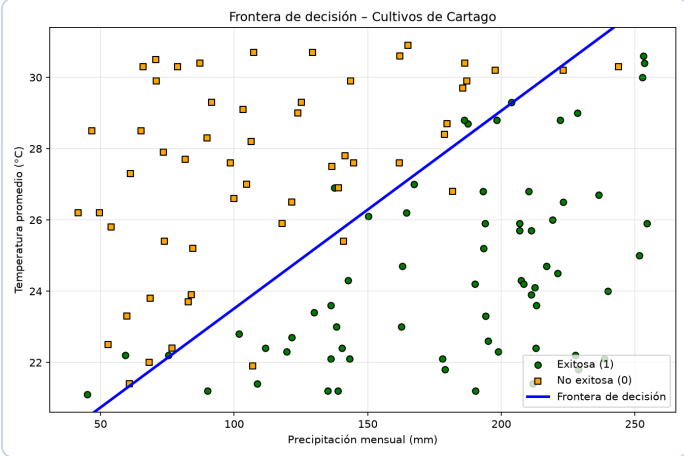


Figura 3: Frontera de decisión lineal en el plano real (frontera_decision.png). Separa con gran fidelidad lotes exitosos (verdes) de no exitosos (naranjas).

Lote Nuevo Evaluado	Probabilidad Estimada	Decisión	Interpretación Agronómica para Cartago
220.0 mm / 23.5 °C	**99.48%**	**EXITOSA**	Condiciones ideales: abundante precipitación y clima templado. Prioridad baja de asistencia.
60.0 mm / 30.0 °C	**0.11%**	**NO EXITOSA**	Déficit hídrico severo combinado con calor extremo. Prioridad crítica urgente de asistencia agronómica.
150.0 mm / 26.0 °C	**55.65%**	**EXITOSA (Dudosa)**	Lote en plena frontera de decisión (incertidumbre). Requiere visita técnica preventiva para desempatar el riesgo.

4. INTERPRETACIÓN AGRONÓMICA Y TÉCNICA

¿Qué significa el signo de w1 y de w2 para un agricultor de Cartago?

El signo refleja la relación biológica aprendida por la neurona:

• **\(w_1 = +2.6217\) (Signo Positivo):** Indica que ante mayor precipitación mensual, el puntaje \(z\) aumenta exponencialmente, elevando la probabilidad de éxito de la cosecha.

• **\(w_2 = -2.3658\) (Signo Negativo):** Indica que temperaturas más elevadas disminuyen el puntaje, actuando como un factor perjudicial (estrés hídrico y térmico).

• **Magnitudes comparables:** Como las variables fueron estandarizadas, comparar sus valores absol (\(2.62\) vs \(2.36\)) demuestra que la lluvia y la temperatura ejercen un peso casi idéntico en el desenlace del cultivo en la zona de Cartago.

¿Qué cambió con el Reto 5 y por qué?

Antes del Reto 5 las características tenían escalas muy dispares (precipitación en centenas, temperatura en decenas). Los gradientes asociados a la lluvia eran enormes, generando pasos gigantescos que catapultaban los pesos hacia valores infinitos e hicieron colapsar la función exponencial (`RuntimeWarning: overflow in exp`), provocando una pérdida aberrante de 5.2830. Al aplicar estandarización Z-score, la superficie de costo se convirtió en un tazón simétrico, permitiendo que el gradiente descendiera uniformemente hacia el mínimo global con pérdida de 0.2281 y 90% de exactitud.

5. EXPERIMENTO FINAL — ANÁLISIS DE HIPERPARÁMETROS Y LOTES ADICIONALES

Experimento Elegido	Qué se Modificó	Resultado Obtenido	Conclusión del Experimento
Experimento C (Lote 4 Adicional) *Principal*	Se evaluó el lote adicional `[100.0 mm, 28.0 °C]` estandarizado con la media y desviación de los 120 lotes.	**Probabilidad: 2.84%** Decisión: **NO EXITOSA** Riesgo: 97.16% de probabilidad de fracaso.	Demuestra la utilidad predictiva: permite a la cooperativa agendar asistencia técnica prioritaria antes de iniciar la siembra para mitigar el calor y sequía.
Experimento A (Tasa de Aprendizaje) *Comparativo*	Se evaluó \(□lpha = 0.001\) y \(□lpha = 1.0\) durante 500 épocas.	Con \(□lpha=0.001\): Pérdida final de 0.6289. Con \(□lpha=1.0\): Pérdida final de 0.2050 (90.00% exactitud).	Con datos estandarizados, una tasa mayor converge sin desbordarse; una tasa ínfima (0.001) aprende con lentitud inaceptable.
Experimento B (Número de Épocas) *Comparativo*	Se comparó el entrenamiento a 20 épocas vs 3000 épocas.	Con 20 épocas: Pérdida 0.5021. Con 3000 épocas: Pérdida 0.2055 (90.83% exactitud).	Entrenar 3000 épocas no aporta ventaja frente a 500 (pérdida 0.2281 vs 0.2055); la curva se aplana después de la época 350.

6. HIPÓTESIS VS. REALIDAD

Lo que creíamos que faltaba (Misión 0)	¿Era necesario? ¿Por qué?
Vectorización en NumPy	**Totalmente indispensable.** Permitió calcular simultáneamente 120 lotes con álgebra matricial, reduciendo el código a `X @ w + b` sin ciclos for lentos.
Función Sigmoide	**Completamente necesaria.** Proporcionó una función continua diferenciable con gradiente no nulo y salidas interpretables como probabilidades del 0 al 100%.
Pérdida por Entropía Cruzada	**Esencial.** Evaluó con precisión la incertidumbre probabilística, castigando asintóticamente las suposiciones equivocadas.
Estandarización Z-Score	**El factor más crítico.** Sin escalado, los gradientes colapsaron la función exponencial y generaron desbordamientos numéricos.

Lo que no nos imaginábamos que faltaba:

No anticipamos que, incluso teniendo matemáticamente impecables la Sigmoide, los Gradientes y la Entropía Cruzada, el modelo empeorara su desempeño si no se normalizaban las entradas antes del entrenamiento. Tampoco teníamos en cuenta que los lotes nuevos no deben estandarizarse calculando una media propia, sino que **deben transformarse estrictamente con la media y desviación del conjunto de entrenamiento** para preservar el espacio dimensional aprendido.

7. CONCLUSIONES Y PREGUNTA ABIERTA

1. **De la neurona lógica al clasificador probabilístico:** El reemplazo de la función escalón de McCulloch-Pitts por la activación Sigmoide convierte al perceptrón en una herramienta rigurosa para la toma de decisiones bajo incertidumbre en entornos productivos reales.
2. **El preprocesamiento es tan determinante como la arquitectura:** La estandarización de variables (Z-score) no es un paso cosmético; es una necesidad matemática indispensable para garantizar una superficie de error simétrica y convergencia sin desbordamientos de punto flotante.
3. **Interpretabilidad agronómica directa:** A diferencia de modelos de caja negra, los pesos del perceptrón brindan interpretabilidad explícita: los signos y magnitudes relativas ($w_1 = +2.62$) y ($w_2 = -2.36$) traducen de forma transparente el impacto de los factores climáticos para los campesinos de la región.

Pregunta que nos queda abierta:

¿Qué ocurriría en un escenario climático no lineal, por ejemplo si un exceso extremo de precipitación (> 300 mm) causara inundación y pudrición de raíces, provocando que tanto poca lluvia como demasiada lluvia arruinen la cosecha? En tal caso, una frontera de decisión lineal simple como la del perceptrón no podría separar las regiones; ¿cuántas neuronas y capas ocultas (MLP) requeriríamos para aprender fronteras curvas cerradas?

ANEXO: CÓDIGO FUENTE COMPLETO Y SALIDA DE CONSOLA

Código Fuente Resuelto: `src/perceptron_reto.py` **(con etiquetas oficiales por rol)**

```
"""
perceptron_reto.py  –  Actividad grupal Clase 9
Construcción completa del perceptrón con descenso del gradiente.

¿CÓMO FUNCIONA ESTE ARCHIVO?
  - El código tiene 7 RETOS marcados con  >>> RETO X <<<.
  - Cada reto tiene un PUNTO DE CONTROL que verifica si quedó bien resuelto.
  - Al ejecutar el script, se detiene en el primer reto pendiente y muestra una pista.
  - Resuelva el reto, guarde (Ctrl + S) y vuelva a ejecutar:  python perceptron_reto.py
  - Cuando todos los retos estén superados, el script genera los gráficos finales.

Genera: curva_aprendizaje.png, frontera_decision.png
"""

import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


def punto_de_control(superado, reto, pista):
    """Verifica un reto. Si no está superado, muestra la pista y detiene el programa."""
    if superado:
        print(f"[OK] {reto} superado")
    else:
        print(f"\n[PENDIENTE] {reto}")
        print(f"   Pista: {pista}")
        print("   Complete el reto, guarde con Ctrl + S y vuelva a ejecutar el script.")
        sys.exit()


# ============================================================
# 1. Cargar los datos
# ============================================================
df = pd.read_csv("../data/cultivos_cartago.csv")
X = df[["precipitacion_mm", "temperatura_c"]].to_numpy()
y = df["cosecha_exitosa"].to_numpy()
print(f"Datos: X tiene forma {X.shape} | y tiene forma {y.shape}\n")


# ============================================================
# >>> RETO 1 <<<  Vectorización: todas las sumas ponderadas a la vez
# ============================================================
def suma_ponderada(X, w, b):
    """Debe devolver z = w1*x1 + w2*x2 + b para TODAS las filas de X, sin usar ciclos."""
    # [Coordinador]
    return X @ w + b


X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
z_and = suma_ponderada(X_and, np.array([0.5, 0.5]), -0.7)
punto_de_control(
    z_and is not None and np.allclose(z_and, [-0.7, -0.2, -0.2, 0.3]),
    "RETO 1 (vectorización)",
    "Con la neurona AND de la Clase 8 (w=[0.5, 0.5], b=-0.7) se esperaba [-0.7 -0.2 -0.2 0.3]. "
    "¿Qué operador de NumPy multiplica una matriz por un vector? (Clase 4)",
)


# ============================================================
# >>> RETO 2 <<<  Una activación que entregue probabilidades
# ============================================================
def sigmoide(z):
```

```
    """Debe convertir cualquier número z en un valor entre 0 y 1 (0.5 cuando z = 0)."""
    # [Coordinador]
    return 1 / (1 + np.exp(-z))


prueba = sigmoide(np.array([-6.0, 0.0, 6.0]))
punto_de_control(
    prueba is not None and np.allclose(prueba, [0.0025, 0.5, 0.9975], atol=1e-3),
    "RETO 2 (función sigmoide)",
    "Para z = -6, 0 y 6 se esperaba aproximadamente 0.0025, 0.5 y 0.9975. "
    "Busque la fórmula de la función sigmoide; en NumPy, e elevado a la x es np.exp(x).",
)


def predecir_probabilidad(X, w, b):
    return sigmoide(suma_ponderada(X, w, b))


# ============================================================
# >>> RETO 3 <<<  Medir qué tan mal predice el modelo
# ============================================================
def calcular_perdida(y_real, y_prob):
    """Pérdida logarítmica (entropía cruzada binaria). Debe devolver UN solo número."""
    epsilon = 1e-9
    y_prob = np.clip(y_prob, epsilon, 1 - epsilon)   # evita log(0)
    # [Verificador]
    return -np.mean(y_real * np.log(y_prob) + (1 - y_real) * np.log(1 - y_prob))


perdida_inicial = calcular_perdida(np.array([1, 0, 1, 0]), np.array([0.5, 0.5, 0.5, 0.5]))
perdida_segura = calcular_perdida(np.array([1]), np.array([0.9]))
punto_de_control(
    perdida_inicial is not None and np.isclose(perdida_inicial, 0.6931, atol=1e-3)
    and np.isclose(perdida_segura, 0.1054, atol=1e-3),
    "RETO 3 (función de pérdida)",
    "Si el modelo dice 0.5 para todo, la pérdida debe ser 0.6931; si acierta con 0.9, debe ser 0.1054. "
    "Busque 'binary cross entropy'. Use np.log y np.mean.",
)


def calcular_exactitud(y_real, y_prob):
    return np.mean((y_prob >= 0.5).astype(int) == y_real)


# ============================================================
# >>> RETO 4 (parte A) <<<  ¿Hacia dónde mover los pesos?
# ============================================================
def calcular_gradientes(X, y_real, y_prob):
    """Debe devolver (gradiente_w, gradiente_b) promediando sobre todos los ejemplos."""
    # [Verificador]
    error = y_prob - y_real
    gradiente_w = (X.T @ error) / len(y_real)
    gradiente_b = np.mean(error)
    return gradiente_w, gradiente_b


gw, gb = calcular_gradientes(np.array([[1.0, 0.0], [0.0, 1.0]]), np.array([1, 0]), np.array([0.5, 0.5]))
punto_de_control(
    gw is not None and np.allclose(gw, [-0.25, 0.25]) and np.isclose(gb, 0.0),
    "RETO 4A (gradientes)",
    "Con X=[[1,0],[0,1]], y=[1,0] y y_prob=[0.5,0.5] se esperaba gradiente_w=[-0.25 0.25] y gradiente_b=0. "
    "El error es (y_prob - y_real); el gradiente de w usa la transpuesta de X y se divide por n.",
)


# ============================================================
# >>> RETO 5 <<<  Preparar las entradas (resuélvalo cuando el punto de control lo pida)
# ============================================================
# [Programador]
media = X.mean(axis=0)
```

```
desviacion = X.std(axis=0)
X_esc = (X - media) / desviacion


# ============================================================
# 2. Entrenamiento con descenso del gradiente
# ============================================================
w = np.zeros(2)
b = 0.0
tasa_aprendizaje = 0.1
epocas = 500
historial_perdida = []
historial_exactitud = []

for epoca in range(1, epocas + 1):
    y_prob = predecir_probabilidad(X_esc, w, b)
    gradiente_w, gradiente_b = calcular_gradientes(X_esc, y, y_prob)

    # >>> RETO 4 (parte B) <<<  Dar el paso de aprendizaje
    # [Programador]
    w = w - tasa_aprendizaje * gradiente_w
    b = b - tasa_aprendizaje * gradiente_b

    if epoca == 1:
        punto_de_control(
            np.allclose(w, -tasa_aprendizaje * gradiente_w) and np.isclose(b, -tasa_aprendizaje * gradiente_b),
            "RETO 4B (actualización de pesos)",
            "Los pesos no cambiaron o se movieron en la dirección equivocada. Recuerde la Clase 8: el nuevo peso "
            "depende del peso anterior, de la tasa de aprendizaje y del gradiente. ¿Se suma o se resta para ir 'cuesta
        )

    historial_perdida.append(calcular_perdida(y, y_prob))
    historial_exactitud.append(calcular_exactitud(y, y_prob))
    if epoca == 1 or epoca % 100 == 0:
        print(f"Época {epoca:>3} | Pérdida: {historial_perdida[-1]:.4f} | "
              f"Exactitud: {historial_exactitud[-1]:.2%} | w=[{w[0]:.3f}, {w[1]:.3f}] b={b:.3f}")


y_prob_final = predecir_probabilidad(X_esc, w, b)
exactitud_final = calcular_exactitud(y, y_prob_final)
print(f"\nRESULTADO: pérdida final {calcular_perdida(y, y_prob_final):.4f} | exactitud {exactitud_final:.2%}")
print(f"Pesos finales: w1={w[0]:.4f}, w2={w[1]:.4f}, b={b:.4f}\n")

punto_de_control(
    media is not None and desviacion is not None
    and np.allclose(X_esc.mean(axis=0), 0, atol=1e-6) and np.allclose(X_esc.std(axis=0), 1, atol=1e-6),
    "RETO 5 (preparación de las entradas)",
    "El modelo entrena, pero ¿los resultados son buenos? ¿Apareció alguna advertencia (RuntimeWarning)? "
    "Compare los valores de la lluvia (40 a 260) con los de la temperatura (21 a 31). "
    "Recuerde lo que se hizo en la Clase 7 antes de entregar datos a un modelo. "
    "Guarde también en 'media' y 'desviacion' los valores usados: se necesitan en los retos 6 y 7.",
)

# ============================================================
# 3. Curva de aprendizaje
# ============================================================
fig, (eje1, eje2) = plt.subplots(1, 2, figsize=(12, 4.5))
eje1.plot(historial_perdida, color="red")
eje1.set_title("Pérdida por época")
eje1.set_xlabel("Época")
eje1.grid(True, alpha=0.3)
eje2.plot(historial_exactitud, color="green")
eje2.set_title("Exactitud por época")
eje2.set_xlabel("Época")
eje2.set_ylim(0, 1.05)
eje2.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("curva_aprendizaje.png", dpi=150)
plt.close()
print("Gráfico guardado: curva_aprendizaje.png")
```

```
# ============================================================
# >>> RETO 6 <<<  La frontera de decisión
# ============================================================
x1_original = np.linspace(X[:, 0].min(), X[:, 0].max(), 100)
x1_esc = (x1_original - media[0]) / desviacion[0]
# La frontera es la línea donde la neurona duda totalmente (probabilidad 0.5).
# [Relator]
x2_esc = -(w[0] * x1_esc + b) / w[1]

punto_de_control(
    x2_esc is not None and np.allclose(w[0] * x1_esc + w[1] * x2_esc + b, 0, atol=1e-6),
    "RETO 6 (frontera de decisión)",
    "¿Para qué valor de z la sigmoide da exactamente 0.5? Escriba esa igualdad con w1, x1, w2, x2 y b, "
    "y despeje x2.",
)
x2_original = x2_esc * desviacion[1] + media[1]

plt.figure(figsize=(9, 6))
plt.scatter(X[y == 1, 0], X[y == 1, 1], color="green", edgecolors="black", label="Exitosa (1)")
plt.scatter(X[y == 0, 0], X[y == 0, 1], color="orange", marker="s", edgecolors="black", label="No exitosa (0)")
plt.plot(x1_original, x2_original, color="blue", linewidth=2.5, label="Frontera de decisión")
plt.ylim(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5)
plt.title("Frontera de decisión – Cultivos de Cartago")
plt.xlabel("Precipitación mensual (mm)")
plt.ylabel("Temperatura promedio (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("frontera_decision.png", dpi=150)
plt.close()
print("Gráfico guardado: frontera_decision.png")

# ============================================================
# >>> RETO 7 <<<  Usar el modelo con lotes nuevos
# ============================================================
lotes_nuevos = np.array([
    [220.0, 23.5],
    [60.0, 30.0],
    [150.0, 26.0],
])
# [Relator]
lotes_nuevos_esc = (lotes_nuevos - media) / desviacion

punto_de_control(
    lotes_nuevos_esc is not None and np.allclose(lotes_nuevos_esc, (lotes_nuevos - media) / desviacion),
    "RETO 7 (predicción de lotes nuevos)",
    "Los lotes nuevos deben pasar por la MISMA preparación que los datos de entrenamiento. "
    "¿Con qué media y qué desviación? ¿Las de los 3 lotes nuevos o las de los 120 lotes?",
)

for (lluvia, temperatura), probabilidad in zip(lotes_nuevos, predecir_probabilidad(lotes_nuevos_esc, w, b)):
    decision = "EXITOSA" if probabilidad >= 0.5 else "NO EXITOSA"
    print(f"Lluvia {lluvia:>5.1f} mm | Temp {temperatura:>4.1f} °C -> {probabilidad:.2%} -> {decision}")

print("\n*** ¡Todos los retos superados! Perceptrón completo construido desde cero. ***")
```

Salida de Consola al Ejecutar `python perceptron_reto.py`

```
Datos: X tiene forma (120, 2) | y tiene forma (120,)

[OK] RETO 1 (vectorización) superado
[OK] RETO 2 (función sigmoide) superado
[OK] RETO 3 (función de pérdida) superado
[OK] RETO 4A (gradientes) superado
[OK] RETO 4B (actualización de pesos) superado
Época   1 | Pérdida: 0.6931 | Exactitud: 54.17% | w=[0.028, -0.025] b=0.004
Época 100 | Pérdida: 0.3183 | Exactitud: 90.83% | w=[1.333, -1.180] b=0.181
```

```
Época 200 | Pérdida: 0.2675 | Exactitud: 90.83% | w=[1.849, -1.651] b=0.237
Época 300 | Pérdida: 0.2467 | Exactitud: 90.83% | w=[2.180, -1.957] b=0.270
Época 400 | Pérdida: 0.2353 | Exactitud: 90.00% | w=[2.426, -2.184] b=0.295
Época 500 | Pérdida: 0.2281 | Exactitud: 90.00% | w=[2.622, -2.366] b=0.315

RESULTADO: pérdida final 0.2281 | exactitud 90.00%
Pesos finales: w1=2.6217, w2=-2.3658, b=0.3152

[OK] RETO 5 (preparación de las entradas) superado
Gráfico guardado: curva_aprendizaje.png
[OK] RETO 6 (frontera de decisión) superado
Gráfico guardado: frontera_decision.png
[OK] RETO 7 (predicción de lotes nuevos) superado
Lluvia 220.0 mm | Temp 23.5 °C -> 99.48% -> EXITOSA
Lluvia  60.0 mm | Temp 30.0 °C -> 0.11% -> NO EXITOSA
Lluvia 150.0 mm | Temp 26.0 °C -> 55.65% -> EXITOSA

*** ¡Todos los retos superados! Perceptrón completo construido desde cero. ***
```